

Reviewer Pack — Minimal Order of Groupoid Representations and Circuit Satisfiability...

Entry 64cbccc70d53 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 13:33:33 UTC

Minimal Order of Groupoid Representations and Circuit Satisfiability Complexity

Entry ID: 64cbccc70d53 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 13:33:33 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): Complexity Theory (Circuit Satisfiability)

Statement:

For every Boolean circuit C with n inputs, the minimal order of groupoid representations required to classify its satisfiability is at most $O(n \log n)$. Equivalently, there exists a constant c such that for all circuits C , $|O(G(C))| \leq c * n \log n$, where $O(G)$ denotes the order of the groupoid representing the circuit G .

Rationale (proposer's reasoning):

Groupoids provide a framework for studying equivalence relations and can be used to represent abstract structures. By exploring the minimal order of groupoid representations needed to classify circuit satisfiability, we may uncover new structural properties that are difficult to achieve with traditional complexity measures. This bridge could potentially reveal hidden connections between category theory and computational complexity.

Taxonomy category: GROUPOID_TO_CIRCUIT_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e91a1438723057c3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal order of groupoid representations $O(G(C))$ for a Boolean circuit C with n inputs will be considered supported if the average correlation coefficient between $|O(G(C))|$ and $n \log n$ across at least 30 random seeds is ≥ 0.8 , with no seed producing an $O(G(C))$ exceeding $c * n \log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_circuit(n):
    if n == 1:
        return [[0, 1]]

    circuit = []
    for _ in range(n - 1):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, len(circuit) - 1) for _ in range(gate_type == 'AND' + 1)]
        output = len(circuit)
        circuit.append([gate_type, inputs, output])

    return circuit

def evaluate_circuit(circuit, assignment):
    stack = []
    for gate in reversed(circuit):
        if gate[0] == 'AND':
            a = assignment[gate[2]]
            b = assignment[gate[1][0]]
            c = assignment[gate[1][1]]
            stack.append(a and (b or c))
        elif gate[0] == 'OR':
            a = assignment[gate[2]]
            b = assignment[gate[1][0]]
            c = assignment[gate[1][1]]
            stack.append(a or (not b and not c))

    return stack[-1]
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        if n > n_max:
            n_max = n

        for _ in range(5):
            circuit = generate_circuit(n)
            assignment = {i: random.randint(0, 1) for i in range(len(circuit))}
            result = evaluate_circuit(circuit, assignment)

            instances_tested += 1
            metric_values.append(result)

    mean_value = sum(metric_values) / len(metric_values)
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / len(metric_values))

    conjecture_holds = all(abs(x - y) <= 0.1 * y for x, y in zip(metric_values, [n * math.log(n) for n in n_values]))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "circuit_satisfiability",
        "metric_value": mean_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")

    mean_value = sum(trial_result["metric_value"] for trial_result in [run_trial(seed) for seed in seeds])
    std_value = math.sqrt(sum((trial_result["metric_value"] - mean_value) ** 2 for trial_result in [run_trial(seed) for seed in seeds]) / len(seeds))

    support_fraction = sum(1 for trial_result in [run_trial(seed) for seed in seeds] if trial_result["conjecture_holds"])

    if all(trial_result["conjecture_holds"] for trial_result in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not trial_result["conjecture_holds"] for trial_result in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed=1")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29cbcd15.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29cbcd15.py", line 60, in run_trial
    circuit = generate_circuit(n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29cbcd15.py", line 25, in generate_circ
    inputs = [random.randint(0, len(circuit) - 1) for _ in range(gate_type == 'AND' + 1)]
                                     ~~~~~^
TypeError: can only concatenate str (not "int") to str
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18799
2	preregistration	ollama_remote	glm4:latest	0	0	9600
3	novelty	ollama_remote	glm4:latest	0	0	9815
4	novelty	ollama_remote	glm4:latest	0	0	8416
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11410
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8805
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11589
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11567
9	judge	ollama_remote	glm4:latest	0	0	14110

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104111 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/64cbccc70d53.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/64cbccc70d53.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/64cbccc70d53.tar.gz` (if generated)